Video Upload System Architecture

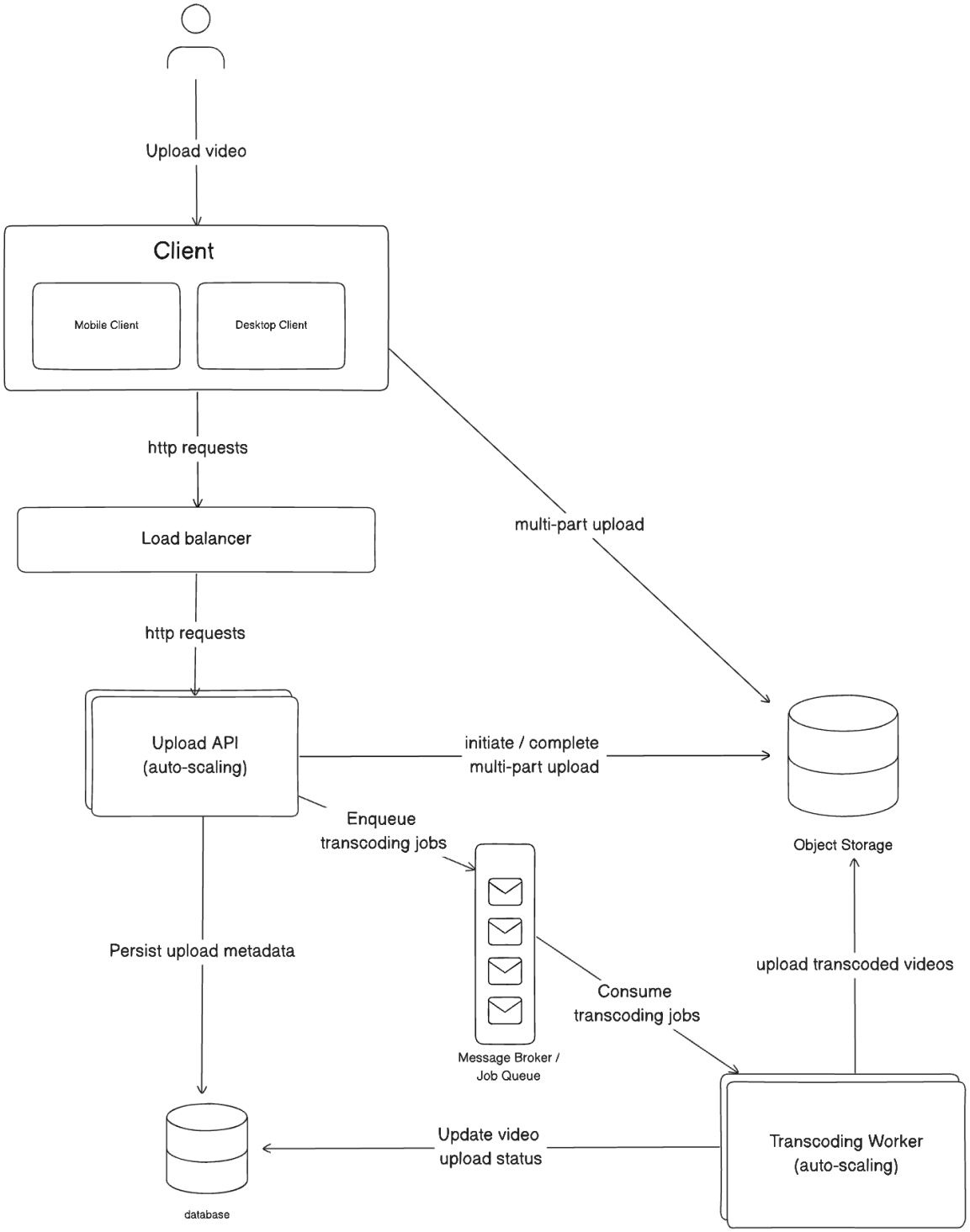

Fig 1. System Architecture Diagram

System Overview

When uploading a video, the client application first initializes an upload session by calling the upload API, which in turn initializes the upload session with the Object Store. The Upload

API then generates presigned URLs for each chunk to be uploaded and returns those to the client. The client can then begin uploading the video chunks to the presigned URLs.
Since we wish to support upload from mobile, the likelihood of interruptions during long running uploads is higher (ex: background app suspension, network issues). With multi-part uploads, in the event of an interruption, we can resume the upload with only the missing chunks as opposed to starting from the beginning.
Upon completing the uploads to the Object Store, the client notifies the upload API, which instructs the Object Store to finalize the upload session, and assemble the video. The upload API can then enqueue a transcoding job in the job queue, which will be picked up by the next available transcoding worker. The worker will first inspect the video (validate supported codec, not corrupted, malware scan) and then generate a configured set of renditions (480p, 720p, 1080p) using FFmpeg. When complete, the worker will upload the renditions to the Object Store and persist their respective metadata to the database.

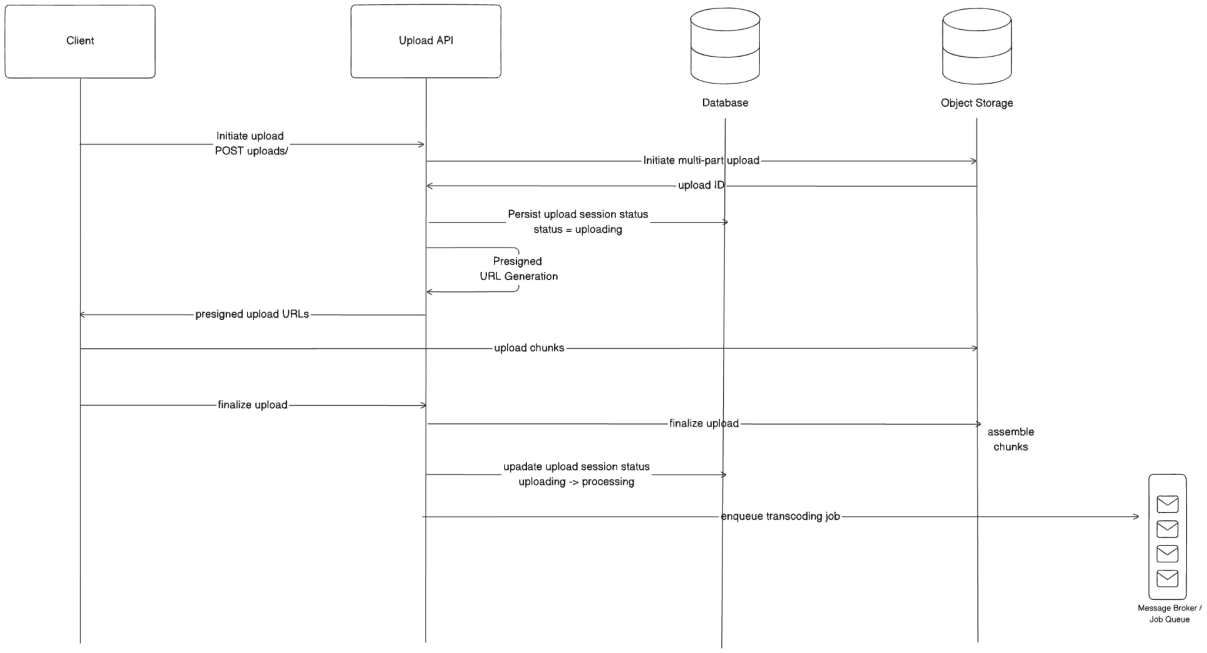

Fig 2. Multi-part Upload System Sequence Diagram

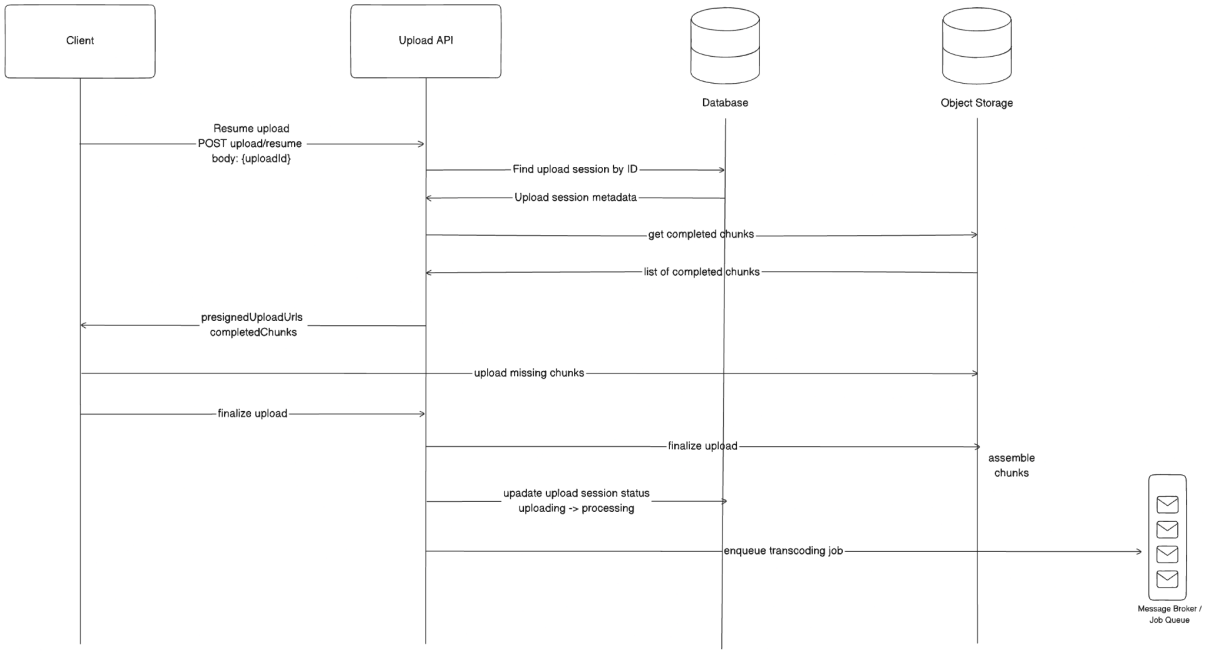

Fig 3. Upload Resume After Interruption System Sequence Diagram

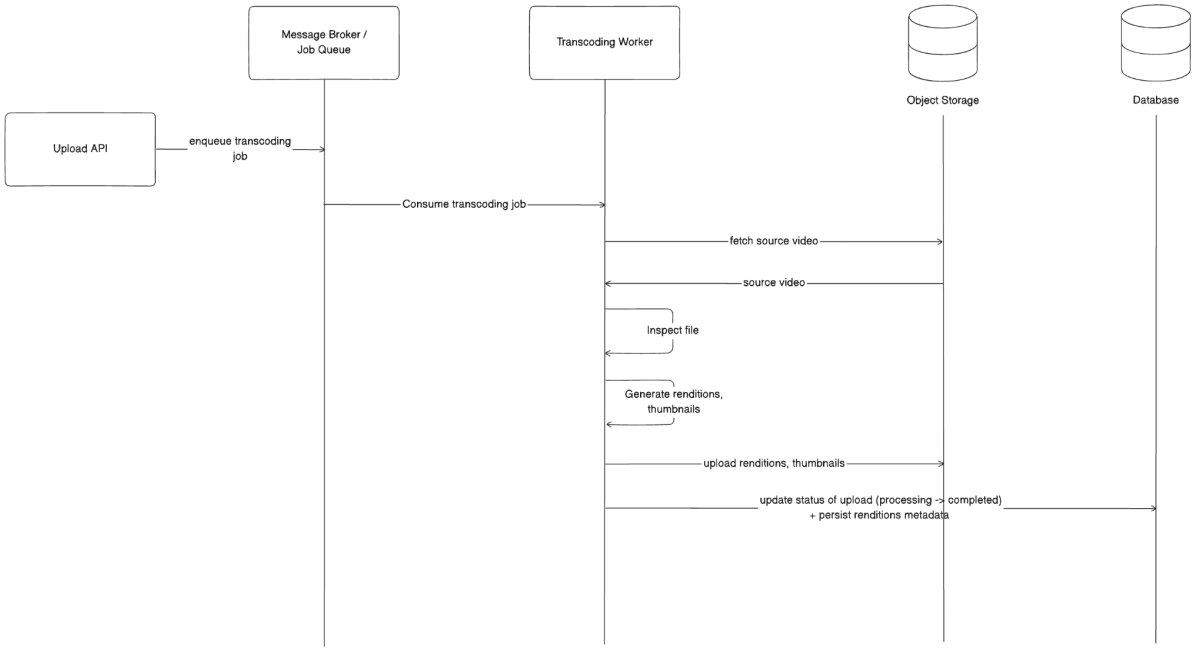

Fig 4. Transcoding Pipeline System Sequence Diagram

Tools and technologies used

Depending on the familiarity of the team, different cloud providers could be used, as main cloud providers have equivalent offerings. For the purposes of this design, we will focus on the AWS ecosystem.
Beginning from the entry point of the system, the Upload API, could be a NodeJS API running on an EC2 instance configured inside a scaling group that could be set up to add instances when the average CPU utilization across instances exceeds 60%. This would ensure the system remains responsive during activity spikes. We could optionally configure a ceiling on the number of instances to avoid incurring excessive costs (depending on what the budget for this application is). This would result in degraded performance in the form of latency for the user during peak usage periods, but would help to control costs.
To spread incoming requests evenly across instances we could use a load balancer like Amazon ELB.
For the transcode worker we can also use a NodeJS server on an EC2 instance configured inside a separate scaling group. In this case the auto-scaling could be configured to depend upon the depth of the message queue, while also having a ceiling on the number of instances to control costs. Transcoding is very computationally demanding and being able to clear a very large queue of jobs would require spawning a very large number of workers, which would incur significant costs. Therefore, in the event of usage spikes, there will most likely be a larger delay before videos are ready for viewing.
The job queue could be implemented using Amazon SQS (simple queue service).

Similarly to the cloud service, the selection of the language/framework in which the Upload API and the Transcoding Worker is implemented should be selected according to team familiarity. The only strong requirement is that the language selected for the worker has a library for working with FFmpeg which we rely on to process user video. We are quite flexible on this since most major languages possess an FFmpeg library.

Data storage and cross-region replication

Our goal is to support usage from both North America and Europe so it would be preferable to have our service running in both regions to keep latency for users low. The primary implication on our system is that we must handle cross-region replication for our database and our Object Storage service. Since we are a smaller team of 5 developers it would be preferable to use managed services that can handle cross-region replication for us. For this reason the database for our Upload API could be an Amazon RDS database with Postgres, configured for cross-region replication. For the Object Storage, Amazon S3 also supports cross-region replication.
For both our main database and the Object Store eventual consistency is the default pattern as cross-region replication is inherently time-consuming and strong consistency is not possible nor would it be necessary, as a small delay between upload and visibility in a different region is acceptable.

Note on use of managed services

Given our team size and target timeline to production, we prefer to rely on managed services as much as possible to increase velocity, reduce maintenance burden. Not only is the team

smaller, but we may not have the expertise to handle cross-region replication for our database for example. This is why we prefer to rely on services like Amazon RDS for our database, S3 for our Object Storage, and Amazon SQS for our job queue. Furthermore, these services have out-of-the-box solutions for observability which are useful for monitoring but also for configuring auto-scaling setups (ex queue-depth on the SQS).

Handling high usage

In our system, the main bottleneck is the transcoding. This process is very compute-intensive, and in moments of peak usage (ex: viral events) unless our budget allows significant spending on worker scaling, we expect a large increase in the delay between upload and the video being available for viewing by users. Depending on the budget, we may increase the ceiling of worker instances if we sought to reduce latency. An alternative would be to have a premium tier for users which would allow for priority processing.

The Upload API would also increase in number of instances during high-load, but we wouldn't expect any persistent increase in latency as it is simply in charge of authentication and authorization, coordinating uploads, retries, and executing simple database operations, which are not computationally expensive operations.

The managed services that we are using, namely the Amazon RDS database, S3 Object Storage, and Amazon SQS would not need any particular consideration with regards to scaling during peak load as we expect them to be able to handle that traffic. With 20-30K DAU, even at peak load, if we assume 10 uploads per user (which is unrealistically high) we would have around 300K items in our queue which would not pose a problem for the managed job queue (SQS), S3 or the managed database (RDS).

Example API for upload process

Initiation of upload

```
POST uploads/initiate
{
      fileName: “concert.mp4”,
      size: 550000000,
      format: “video/mp4”
}

Response
{
      uploadSessionId: “upload-001”,
      status: “uploading”,
      chunkSize: 5242880, // = 5 * 1024 * 1024 = 5MB
      totalChunks: 11
}
```

Get signed urls

```
POST uploads/{uploadSessionId}/get-urls
{
      parts: [1, 2, 3, 4, 5]
}

Response
{
      uploadUrls: [
          {
             part: 1,
             uploadUrl: “...”
          },
          {
             part: 2,
             uploadUrl: “...”
          },
          ...
      ]
}
```

Get upload status

Used when resuming an interrupted upload. Using this information the client can once again request the URLs for the missing chunks.

```
GET uploads/{uploadSessionId}/status
{
      uploadSessionId: “upload-001”
}

Response
{
      status: “uploading”,
      completedChunks: [1, 2, 4],
      totalChunks: 11,
      chunkSize: 5000000
}
```

Completing upload

```
POST uploads/{uploadSessionId}/complete
{
      uploadSessionId: “upload-001”
}
```

The Upload API would then query the Object Storage to get the etags of the completed parts, and finalize the multi-part upload by calling an upload finalization endpoint on the Object Storage service, passing the manifest in the payload (the manifest would be a list of objects that has the part number and the respective etag).

```
Response
{
      uploadSessionId: “upload-001”,
      status: “processing”
}
```